

Bus Tracking Project – Knowledge Transfer

Objective:

Build a scalable real-time bus tracking system for 25+ buses that: - Tracks bus location via GPS - Sends automatic notifications to students based their bus stop - Uses Leaflet with React frontend (no manual map clicking is required).

Hardware Setup:

Each bus will have: - ESP32 + GPS Module to fetch latitude/longitude

Components used -

1. ESP32 Development Board
2. NEO-6M GPS Module
3. SIM800L GSM Module
4. Push Button (for SOS trigger)
5. Li-ion Battery or 5V Power Supply
6. Voltage Regulator (Optional, for SIM800L)
7. Breadboard
8. Jumper Wires
9. GPS Antenna (Improves GPS signal reception, especially in weak signal areas.)
10. Level Shifter(Needed when working with 5V devices or when there's voltage mismatch)
- 11.MPU6050(for accident detection)
12. LED Light (for visual indicators or emergency status)

What Has Been Done

- ESP32 setup completed.
- MPU6050 wired and basic code uploaded.
- GPS module connected to ESP32.
- Code for each module is available (MPU6050 and GPS).
- Initial testing done for both sensors individually.

ESP32 Setup:

- ESP32 connected to laptop via USB.
- Installed required drivers and Arduino IDE setup.
- ESP32 DEV MODULE (COM3)

Code :

```
#define LED_PIN 2

void setup() {

  pinMode(LED_PIN, OUTPUT);

}

void loop() {

  digitalWrite(LED_PIN, HIGH); // Turn LED ON

  delay(500);

  digitalWrite(LED_PIN, LOW); // Turn LED OFF

  delay(500);

}
```

MPU6050- Accident detect and sos trigger with led

- Install “**MPU6050 by Electronic Cats**” (recommended).
- It will also install **I2Cdev** dependency.

Wiring (ESP32 ↔ MPU6050)

MPU6050 Pin	ESP32 Pin
VCC	3.3V
GND	GND
SDA	D21 (GPIO21)
SCL	D22 (GPIO22)

Test Code :

```
#include <Wire.h>
```

```
#include <MPU6050.h>
```

```
MPU6050 mpu;
```

```
#define SOS_LED 2    // Onboard LED for SOS
```

```
#define ACC_THRESHOLD 15000 // Threshold for accident detection (adjust if needed)
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    Wire.begin(21, 22); // SDA=21, SCL=22
```

```
    pinMode(SOS_LED, OUTPUT);
```

```
    digitalWrite(SOS_LED, LOW); // LED OFF initially
```

```
    Serial.println("Initializing MPU6050...");
```

```
    mpu.initialize();
```

```
    if (mpu.testConnection()) {
```

```
        Serial.println("MPU6050 connection successful ✅");
```

```
    } else {
```

```
        Serial.println("MPU6050 connection failed ❌");
```

```

}

}

void loop() {

    int16_t ax, ay, az;

    int16_t gx, gy, gz;

    // Read accelerometer and gyroscope data

    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    Serial.print("aX = "); Serial.print(ax);

    Serial.print(" | aY = "); Serial.print(ay);

    Serial.print(" | aZ = "); Serial.print(az);

    Serial.print(" | gX = "); Serial.print(gx);

    Serial.print(" | gY = "); Serial.print(gy);

    Serial.print(" | gZ = "); Serial.println(gz);

    // Accident detection: trigger SOS if acceleration exceeds threshold

    if (abs(ax) > ACC_THRESHOLD || abs(ay) > ACC_THRESHOLD || abs(az) > ACC_THRESHOLD) {

        Serial.println("!!! ACCIDENT DETECTED !!! Sending SOS...");

        digitalWrite(SOS_LED, HIGH); // Turn LED ON

        delay(5000);                // Keep LED ON for 5 seconds

        digitalWrite(SOS_LED, LOW); // Turn LED OFF

    }

    delay(500); // half-second delay

}

```

Reads accelerometer (ax, ay, az) and gyroscope (gx, gy, gz) data every 500ms.

Checks if any acceleration value exceeds ACC_THRESHOLD.

If yes:

- Prints "ACCIDENT DETECTED" in Serial Monitor.
- Turns SOS LED ON for 5 seconds.

MPU6050 I2C Connection: intermittent detection issues.

SOS / Accident Detection

- Planned: Detect sudden acceleration or collision via MPU6050 and send an **automatic SOS**.
- Current: Code partially set up, but dependent on stable MPU6050 readings.

Problems / Next Steps:

- Fix MPU6050 I2C issue first.
- Once stable, implement **threshold-based detection** for sudden movements.
- Trigger automatic SOS alerts via push notification or server.

GPS NEO-6M

- TinyGPS++

Wiring ESP32 ↔ Neo-6M

Neo-6M Pin	ESP32 Pin
VCC	3.3V (or 5V — module usually supports both, check your board)
GND	GND
TX	D16 (GPIO16, RX pin on ESP32 side)
RX	D17 (GPIO17, TX pin on ESP32 side)

Code :

```
#include <TinyGPS++.h>

#include <HardwareSerial.h>

TinyGPSPlus gps;

HardwareSerial SerialGPS(1);

void setup() {

  Serial.begin(115200);

  SerialGPS.begin(9600, SERIAL_8N1, 16, 17); // RX=16, TX=17


  Serial.println("GPS Test Starting...");

}

void loop() {

  while (SerialGPS.available() > 0) {

    if (gps.encode(SerialGPS.read())) {

      if (gps.location.isValid()) {

        Serial.print("Latitude: ");

        Serial.println(gps.location.lat(), 6);

        Serial.print("Longitude: ");
```

```
Serial.println(gps.location.lng(), 6);

} else {

    Serial.println("Waiting for GPS signal...");

}

}

}

}
```

Working -

- Upload this sketch.
- Open **Serial Monitor** at **115200 baud**.
- Place GPS antenna **near a window or outside**

At first, you'll see:

- "Waiting for GPS signal..." (can take 1-2 mins).

Once locked, you'll get:

- Latitude: 17.385044
- Longitude: 78.486671 (your location).

Problems to Recover / Fix

1. **MPU6050 I2C Connection**: intermittent detection issues.
2. **GPS Data Parsing**: output shows blocks, needs proper parsing.
3. **Automatic SOS**: cannot be tested until MPU6050 is stable.

Next Steps

1. Stabilize **MPU6050 I2C connection** (hardware & software fixes).
2. Implement **GPS parsing logic** to get proper latitude & longitude.
3. Integrate **automatic SOS detection** using MPU6050.
4. Combine **GPS + SOS logic** to have real-time emergency alerts.

5. Test system in a real-world scenario (bus moving).